

并行计算笔记整理(下)(gpt5.5 plus)

主讲：陈国良教授 原课程链接：[超算学堂EasyHPC 并行计算 - 结构·算法·编程](#)

PC12

1. 第十二章总体：并行程序设计基础

这份 **PC12.pdf** 开始进入第四篇“并行程序设计”的基础部分，主题是 **第十二章：并行程序设计基础**。这一章主要讲并行程序为什么难、并行语言如何构造、并行性与通信/同步问题、五种并行编程风范，以及四种并行程序设计模型。

1.1 本章结构

课件第 2 页列出本章六节：

- 12.1 并行程序设计概述
- 12.2 进程
- 12.3 线程
- 12.4 同步
- 12.5 通信
- 12.6 并行程序设计模型

其中，课件前半部分重点展开 **12.1 并行程序设计概述**，后半部分给出进程、线程、同步、通信的目录页，并重点补充 **12.6 并行程序设计模型**。

2. 12.1 并行程序设计概述

课件第 3 页列出这一节的主要内容：

- 1 并行程序设计难的原因
- 2 并行语言的构造方法
- 3 并行性问题
- 4 交互/通信问题
- 5 五种并行编程风范
- 6 计算圆周率的样本程序

这一节的核心是：

并行程序设计不仅是“把程序分给多个处理器执行”，还必须处理任务划分、数据分配、通信、同步、互斥、负载均衡和可移植性等问题。

3. 并行程序设计难的原因

3.1 技术先行，理论指导不足

课件第 4 页首先指出，并行程序设计长期存在一个问题：

技术先行，缺乏理论指导

也就是说，并行硬件、并行机、集群和编程接口发展很快，但如何系统地、可移植地、可扩展地写并行程序，并没有像顺序程序设计那样成熟。

顺序程序设计中，程序员主要关心：

算法
数据结构
控制流程

而并行程序设计还要额外关心：

任务如何切分
数据放在哪里
进程之间如何通信
什么时候同步
如何避免互斥冲突
怎样保持负载均衡

3.2 语法和语义复杂

并行程序的语法和语义比顺序程序复杂得多。

课件列出程序员需要自己处理的问题：

任务/数据的划分/分配
数据交换
同步和互斥
性能平衡

例如一个普通循环：

```
for (i = 0; i < N; i++)  
    A[i] = B[i] + C[i];
```

顺序程序只需要考虑计算是否正确。

但并行程序要考虑：

每个处理器算哪些 i？
A、B、C 分布在哪些处理器？
B[i] 和 C[i] 是否本地可用？
是否需要通信？
是否有多个处理器同时写 A[i]？
各处理器工作量是否均衡？

所以并行程序的复杂性来自 **正确性 + 性能 + 数据分布 + 通信同步** 的组合。

3.3 可扩展性和异构性问题

课件指出，并行语言常常缺乏：

代可扩展
异构可扩展

可以理解为：

- **代可扩展**：程序能否适应新一代硬件；
- **异构可扩展**：程序能否适应不同类型的机器，例如 CPU 集群、向量机、共享内存机、分布式内存机等。

如果并行程序和某一种机器绑定太紧，那么换平台时可能要大规模重写代码。

这就是并行程序移植困难的原因。

3.4 工具和环境不成熟

课件还指出，并行环境和工具缺乏较长的生长期。

顺序程序有成熟的：

编译器
调试器
性能分析器
IDE
库函数
运行时系统

但并行程序调试更困难，因为它会遇到：

死锁
数据竞争
非确定性执行
通信错误
负载不均衡
同步等待

所以并行程序设计难，不只是语言难，而是整个生态都更复杂。

4. 并行语言的构造方法

课件第 5 页讲三种构造并行语言或并行程序的方法。

4.1 使用库例程构造并行程序

第一种方法是在顺序语言中调用并行库。

例如原来的顺序代码是：

```
for (i = 0; i < N; i++)  
    A[i] = b[i] * b[i+1];  
  
for (i = 0; i < N; i++)  
    c[i] = A[i] + A[i+1];
```

用库例程并行化时，可以写成：

```
id = my_process_id();  
p = number_of_processes();  
  
for (i = id; i < N; i = i + p)  
    A[i] = b[i] * b[i+1];  
  
barrier();  
  
for (i = id; i < N; i = i + p)  
    c[i] = A[i] + A[i+1];
```

这里：

```
id 表示当前进程编号  
p  表示进程总数  
barrier 表示同步路障
```

每个进程只处理：

```
i = id, id+p, id+2p, ...
```

这种方式对应：

```
MPI  
PVM  
Pthreads
```

4.2 库例程方法的特点

优点：

```
容易实现  
不需要新编译器  
可以嵌入 C、Fortran 等已有语言
```

缺点：

```
编译器不知道这些库调用的并行语义  
难以自动检查错误  
难以自动分析和优化
```

例如 MPI 程序中的 `send/recv` 是否匹配，很多时候编译器无法静态检查，只能运行时发现。

4.3 扩展串行语言

第二种方法是直接扩展串行语言，让语言本身支持并行结构。

课件例子是 Fortran 90 的数组语句：

```
A(0:N-1) = b(0:N-1) * b(1:N)
c = A(0:N-1) + A(1:N)
```

这种写法把整个数组作为对象操作。

它表达的是：

对数组中所有元素执行相同或类似操作

这就是数据并行思想。

4.4 扩展语言方法的特点

优点：

编译器能理解并行语义
可以做检查、分析和优化
代码更接近数学表达

缺点：

实现困难
需要新编译器
语言和生态迁移成本较高

4.5 加编译注释构造并程序序

第三种方法是在普通顺序语言中加入编译器注释或指令。

课件例子类似：

```
#pragma parallel
#pragma shared(A,b,c)
#pragma local(i)
{
    #pragma pfor iterate(i=0;N;1)
    for (i=0; i<N; i++)
        A[i] = b[i] * b[i+1];

    #pragma synchronize

    #pragma pfor iterate(i=0;N;1)
    for (i=0; i<N; i++)
        c[i] = A[i] + A[i+1];
}
```

这里的意思是：

parallel：进入并行区域
shared：声明共享变量
local：声明私有变量
pfor：并行循环
synchronize：同步

课件举的例子是：

SGI Power C
HPF

现代 OpenMP 也属于类似思想。

4.6 三种方法比较

课件第 6 页给出表格，可以整理为：

方法	实例	优点	缺点
库例程	MPI, PVM	易于实现，不需要新编译器	无编译器检查、分析和优化
扩展语言	Fortran 90	允许编译器检查、分析和优化	实现困难，需要新编译器
编译器注释	SGL Power C, HPF	介于库例程和扩展语言之间	在普通串行平台上不起作用

可以这样记：

库例程：最实用，但编译器不懂
扩展语言：表达最好，但实现最难
编译注释：折中方案

5. 并行性问题

课件第 7 页开始讲并行性问题。

主要包括：

进程的同构性
静态和动态并行性
进程编组
划分与分配
并行度与粒度

6. 进程的同构性

6.1 SIMD

SIMD 是：

Single Instruction Multiple Data
单指令流多数据流

课件定义为：

所有进程在同一时间执行相同的指令。

这适合数据并行问题，例如：

所有处理器同时执行 $A[i] = B[i] + C[i]$

不同处理器处理不同数据，但执行同样操作。

6.2 MIMD

MIMD 是：

Multiple Instruction Multiple Data
多指令流多数据流

课件定义为：

各个进程在同一时间可以执行不同的指令。

MIMD 更灵活，可以表达任务并行、功能并行和控制并行。

MIMD 又常分为：

SPMD
MPMD

6.3 SPMD

SPMD 是:

Single Program Multiple Data
单程序多数据

课件说明:

各个进程是同构的, 多个进程对不同数据执行相同代码。

SPMD 常对应:

并行循环
数据并行结构
单代码

例如:

```
pid = my_process_id();  
numproc = number_of_processes();  
  
for (i = pid; i <= N; i += numproc)  
    foo(i);
```

所有进程运行同一个程序, 只是 `pid` 不同, 所以处理的数据不同。

6.4 MPMD

MPMD 是:

Multiple Program Multiple Data
多程序多数据

课件说明:

各个进程是异构的, 多个进程执行不同代码。

MPMD 常对应:

任务并行
功能并行
控制并行
并行块
多代码

例如:

```
进程 1 做输入  
进程 2 做计算  
进程 3 做输出
```

这三个进程执行的是不同程序或不同代码段。

6.5 为什么 SPMD 更常用

课件第 7 页特别强调:

要为有 1000 个处理器的计算机编写一个完全异构的并行程序是很困难的。

也就是说, 如果每个处理器都写不同代码, 程序很难维护、调试和扩展。

所以大规模并行程序通常采用 SPMD:

一份代码
多个进程
根据 `rank/id` 决定自己做什么

MPI 程序通常就是 SPMD 风格。

7. 并行块与并行循环

7.1 并行块

课件第 8 页给出并行块形式：

```
parbegin S1 S2 S3 ... Sn parend
```

其中：

`S1, S2, ..., Sn` 可以是不同代码

这更接近 MPMD 或任务并行。

例如：

```
S1: 读数据  
S2: 计算  
S3: 写结果
```

7.2 并行循环

如果并行块中的所有进程共享相同代码，就可以简化为并行循环：

```
parfor (i = 1; i <= n; i++)  
    S(i)
```

这更接近 SPMD 或数据并行。

例如：

```
parfor i = 1 to n:  
    C[i] = A[i] + B[i]
```

并行循环是并行程序中最常见、最容易理解的一种结构。

8. SPMD 程序的构造方法

8.1 用单代码方法说明 SPMD

课件第 9 页给出例子。

目标是表达：

```
parfor (i = 0; i <= N; i++)  
    foo(i)
```

用户实际可以写成：

```
pid = my_process_id();  
numproc = number_of_processes();  
  
for (i = pid; i <= N; i += numproc)  
    foo(i);
```

编译后生成一个可执行程序 `A`，然后通过脚本加载到多个处理结点上：

```
run A -numnodes N
```

每个进程运行同一个程序，但 `pid` 不同，所以自动分担循环迭代。

8.2 用数据并行程序构造 SPMD

课件也给出数据并行风格。

想表达：

```
parfor (i = 0; i <= N; i++)  
    C[i] = A[i] + B[i]
```

可以直接写成：

```
C = A + B
```

或者：

```
forall (i = 1, N)  
    C[i] = A[i] + B[i]
```

这种写法让程序员不用显式写进程编号，数据分布和并行执行由编译器或运行时处理。

9. MPMD 程序的构造方法

9.1 用多代码方法说明 MPMD

课件第 10 页给出 MPMD 例子。

目标是表达：

```
parbegin S1 S2 S3 parend
```

如果语言不支持并行块，用户需要写三个程序：

```
S1  
S2  
S3
```

分别编译成三个可执行程序，然后用脚本加载到不同处理结点上：

```
run S1 on node1  
run S2 on node2  
run S3 on node3
```

这些程序是顺序语言程序，但加上交互库调用。

9.2 用 SPMD 伪造 MPMD

课件还说明，可以用 SPMD 模拟 MPMD：

```
parfor (i = 0; i < 3; i++) {  
    if (i == 0) S1;  
    if (i == 1) S2;  
    if (i == 2) S3;  
}
```

也就是说，所有进程运行同一个程序，但根据进程编号执行不同分支。

课件结论是：

对于可扩展并行机来说，只要支持 **SPMD** 就足够了。

因为 MPMD 可以用 SPMD 伪造出来，而 SPMD 更容易扩展到大量处理器。

10. 静态和动态并行性

10.1 程序结构

课件第 11 页定义程序结构：

程序结构是由程序的组成部分构成程序的方法。

并行程序结构可以是静态的，也可以是动态的。

10.2 静态并行性

静态并行性是指：

程序结构以及进程个数在运行之前就能确定。

例如：

```
parbegin P, Q, R parend
```

这里 **P**，**Q**，**R** 都是静态确定的。

静态并行性通常在：

编译时
连接时
加载时

就能确定。

优点是容易分析、调度和优化。

10.3 动态并行性

动态并行性是指：

进程需要在运行时创建和终止。

课件例子：

```
while (C > 0) begin
    fork(foo(C));
    C := boo(C);
end
```

每次循环都可能创建新进程，进程数量取决于运行时变量 **C**。

动态并行适合：

搜索问题
递归任务
自适应网格
动态负载不均问题

但它更难调度和分析。

11. Fork/Join

11.1 Fork

课件第 12 页说明：

Fork: 派生一个子进程

例如：

`fork(B)`

表示当前进程创建子进程 B。

11.2 Join

课件说明：

Join: 强制父进程等待子进程

例如：

`join(C)`

表示当前进程必须等待 C 完成后才能继续。

11.3 Fork/Join 例子

课件给出三个进程：

```
Process A:
begin
  Z := 1
  fork(B)
  T := foo(3)
end
Process B:
begin
  fork(C)
  X := foo(Z)
  join(C)
  output(X + Y)
end
Process C:
begin
  Y := foo(Z)
end
```

执行关系是：

```
A 创建 B
B 创建 C
B 计算 X
C 计算 Y
B 等待 C 完成
B 输出 X + Y
```

这个例子体现了动态并行程序的基本结构。

12. 进程编组

12.1 进程编组目的

课件第 13 页讲进程编组。

目的：

支持进程间交互

通常把需要频繁交互的进程调度到同一组中。

例如一个并行矩阵乘法中，同一行处理器经常广播 A 块，同一列处理器经常移动 B 块，那么可以按行、列建立进程组。

12.2 进程组成员标识

课件指出，一个进程组成员由以下二者唯一确定：

组标识符
成员序号

可以理解为：

进程属于哪个组
在组内编号是多少

MPI 中的 communicator 和 rank 就很接近这种思想。

13. 划分与分配

13.1 基本原则

课件第 13 页给出原则：

使系统大部分时间忙于计算，而不是闲置或忙于交互；同时不牺牲并行性。

这句话非常关键。

并行程序设计不是让进程越多越好，而是要让处理器尽量做有效计算。

如果处理器大部分时间在：

等待
通信
同步
抢锁
空闲

那么并行程序性能就会很差。

13.2 划分

课件定义：

划分：切割数据和工作负载

例如：

把数组分成 p 段
把矩阵分成 p 个块
把搜索树分成多个子树
把网格分成多个区域

13.3 分配

课件定义：

分配：将划分好的数据和工作负载映射到计算结点或处理器上

例如：

第 0 段给 P0
第 1 段给 P1
...

划分关注“怎么切”，分配关注“放到哪里”。

13.4 显式分配和隐式分配

课件列出两种分配方式：

显式分配
隐式分配

显式分配：

由用户指定数据和负载如何加载

例如 MPI 程序中，程序员明确写：

rank 0 处理第 0 块
rank 1 处理第 1 块

隐式分配：

由编译器和运行时支持系统决定

例如数据并行语言中，程序员写数组操作，具体数据放到哪里由系统安排。

13.5 就近分配原则

课件强调：

进程所需的数据靠近使用它的进程代码

这就是局部性原则。

如果进程经常访问远程数据，会导致大量通信。

所以好的并行程序应尽量做到：

谁用数据，数据就放在谁附近

14. 并行度与并行粒度

14.1 并行度 DOP

课件第 14 页定义：

并行度 Degree of Parallelism, DOP

含义是：

同时执行的分进程数

例如某时刻有 32 个任务同时执行，则并行度约为 32。

14.2 并行粒度 Granularity

课件定义：

并行粒度：两次并行或交互操作之间所执行的计算负载

可以理解为：

每个任务干多少活，才进行一次通信或同步

粒度越细，任务越小，通信同步越频繁。

粒度越粗，任务越大，通信同步相对较少，但并行度可能降低。

14.3 粒度层次

课件列出：

指令级并行
块级并行
进程级并行
任务级并行

可以理解为从细到粗：

粒度	说明
指令级并行	CPU 内部多条指令并行
块级并行	基本块或代码块并行
进程级并行	多进程同时执行
任务级并行	大任务或功能模块并行

14.4 并行度和粒度的关系

课件指出：

并行度与并行粒度大小常互为倒数

也就是：

增大粒度会减小并行度
增加并行度会增加系统同步开销

这就是并行程序设计的经典矛盾：

任务太细：并行度高，但开销大
任务太粗：开销小，但并行度低

好的并行程序要在两者之间折中。

15. 交互 / 通信问题

课件第 15 页开始讲交互。

15.1 交互的定义

课件定义：

交互：进程间的相互影响

进程之间不是孤立执行的，它们可能通过通信、同步、聚集等方式互相影响。

15.2 通信

课件定义通信为：

两个或多个进程间传送数的操作

通信方式包括：

共享变量
父进程传给子进程
消息传递

对应不同体系结构：

共享变量 → 共享内存
父子参数传递 → `fork/join`
消息传递 → 分布式内存 `MPI/PVM`

16. 同步

16.1 同步定义

课件第 16 页定义同步为：

导致进程间相互等待或继续执行的操作

同步的作用是协调进程执行顺序，防止错误访问或不一致状态。

16.2 同步方式

课件列出三类：

原子同步
控制同步
数据同步

16.3 原子同步

原子同步保证一段操作不可分割。

课件例子：

```
parfor (i := 1; i < n; i++) {  
    atomic {  
        x := x + 1;  
        y := y - 1;  
    }  
}
```

这里 `x := x + 1` 和 `y := y - 1` 必须作为整体执行，不能被其他进程插入。

16.4 控制同步：路障

路障同步 barrier 的例子：

```
parfor (i := 1; i < n; i++) {  
    pi  
    barrier  
    qi  
}
```

含义是：

所有进程必须完成 `pi`
才能继续执行 `qi`

barrier 用于阶段式并行程序。

16.5 控制同步：临界区

临界区例子：

```
parfor (i := 1; i < n; i++) {  
    critical {  
        x := x + 1;  
        y := y + 1;  
    }  
}
```

同一时刻只允许一个进程进入 `critical` 区域。

它用于保护共享变量。

16.6 数据同步：锁和信号量

课件给出锁的例子：

```
parfor (i := 1; i < n; i++) {  
    lock(s);  
    x := x + 1;  
    y := y - 1;  
    unlock(s);  
}
```

这里 `s` 是锁或信号量。

作用是：

进入前加锁
执行共享数据操作
结束后解锁

17. 聚集 aggregation

17.1 聚集定义

课件第 17 页定义聚集：

用一串超步将各分进程计算所得的部分结果合并为一个完整的结果，每个超步包含一个短的计算和一个简单的通信或同步。

也就是说，聚集是把多个局部结果合并成全局结果。

17.2 聚集方式

课件列出两类：

归约 `reduce`
扫描 `scan`

17.3 归约 `reduce`

归约把多个值合成一个值。

例如：

```
sum = x1 + x2 + ... + xn  
max = max(x1, x2, ..., xn)
```

17.4 扫描 `scan`

扫描得到所有前缀结果。

例如输入：

1, 3, 5

扫描和结果是：

1, 4, 9

因为：

```
1  
1+3  
1+3+5
```

17.5 向量内积例子

课件给出计算两个向量内积的例子：

```
parfor (i := 1; i < n; i++) {  
    x[i] := A[i] * B[i];  
    inner_product := aggregate_sum(x[i]);  
}
```

过程是：

每个进程先算局部乘积
然后通过 `aggregate_sum` 求总和

这就是典型的归约。

18. 交互的方式

18.1 进程相对于交互代码的状态

课件第 18 页定义进程相对于交互代码 `C` 的四种状态：

到达 `arrived`
在内 `in`
完成 `finished`
在外 `out`

含义如下：

状态	含义
arrived	进程刚到达交互代码 C，但还未进入
in	进程正在交互代码中
finished	进程刚完成交互代码，但还未离开
out	进程不在交互代码中，可能未到达或已离开

18.2 同步交互

课件定义：

同步交互：所有参与者同时到达并执行交互代码 C

也就是说，进程需要等待其他参与者。

典型例子：

同步发送/接收
barrier

18.3 异步交互

课件定义：

异步交互：进程到达 C 后，不必等待其他进程到达即可执行 C

例如非阻塞发送/接收。

异步交互可以减少等待，但程序更难分析。

19. 交互方式与入口/出口条件

课件第 19 页给出一个表格，比较不同交互方式的进入和退出条件。

19.1 同步发送/接收

入口条件：

当事进程 arrived
其他进程 arrived

出口条件：

当事进程 finished
其他进程 finished

含义是双方都到达后才能完成。

19.2 锁定发送

课件说明：

锁定的发送：消息已发完，但不一定已收到

发送方完成发送动作即可继续，不保证接收方已经处理。

19.3 锁定接收

课件说明：

锁定的接收：消息已收到

接收方必须真正收到消息后才能继续。

19.4 非锁定发送/接收

课件说明：

非锁定的发/收：只是发出发/收的请求

也就是非阻塞操作。

发出请求后，进程可以继续执行其他计算。

19.5 路障

barrier 的特点是：

所有进程都必须到达
完成后才能继续

其他进程可能处于 `in` 或 `finished` 状态。

19.6 临界区

临界区要求：

其他进程在外 `out`

也就是说，同一时刻只能有一个进程进入临界区。

20. 交互的模式

课件第 20 页讲交互模式。

20.1 静态交互和动态交互

按交互模式是否能在编译时确定，可以分为：

静态交互
动态交互

静态交互：

通信对象、通信模式在编译时或运行前可确定

动态交互：

通信对象或通信量运行时才确定

20.2 按发送者和接收者数量分类

课件列出：

一对一: point to point
一对多: broadcast, scatter
多对一: gather, reduce
多对多: total exchange, scan, permutation/shift

这些就是并行程序最常见的通信模式。

21. 常见通信模式图解

课件第 21 和 22 页用三个处理器 P1, P2, P3 画了通信模式。

21.1 点到点 point-to-point

例子：

P1 发送一个值给 P3

通信关系是一对一。

21.2 广播 broadcast

例子：

P1 发送一个值给全体

所有处理器收到同一个值。

21.3 播撒 scatter

例子：

P1 向每个结点发送一个不同值

和广播不同，scatter 给不同处理器的数据不同。

21.4 收集 gather

例子：

P1 从每个结点接收一个值

多个处理器的数据汇集到一个处理器。

21.5 全交换 total exchange

例子：

每个结点向每个结点发送一个不同消息

这是通信量最大的模式之一。

矩阵转置、并行排序中的数据重分布常常会产生全交换。

21.6 移位 permutation / shift

例子：

每个结点向下一个结点发送一个值
并接收来自上一个结点的一个值

这在环形通信、流水线、Cannon 矩阵乘法中很常见。

21.7 归约 reduce

课件例子：

P1 得到 $1 + 3 + 5 = 9$

多个值合并成一个值。

21.8 扫描 scan

课件例子：

P1 得到 1
P2 得到 $1 + 3 = 4$
P3 得到 $1 + 3 + 5 = 9$

扫描得到的是前缀结果，不只是一个总结果。

22. 五种并行编程风范

课件第 23 页列出五种并行编程风范：

相并行 Phase Parallel
分治并行 Divide and Conquer Parallel
流水线并行 Pipeline Parallel
主从并行 Master-Slave Parallel
工作池并行 Work Pool Parallel

这些是并行程序组织结构的常见模式。

23. 相并行 Phase Parallel

23.1 基本思想

课件第 24 页讲相并行。

它由一组超级步组成：

计算
同步交互
计算
同步交互
...

每个相内部，各进程独立计算；相与相之间进行通信和同步。

这和 BSP 模型非常接近。

23.2 特点

课件列出：

一组超级步
步内各自计算
步间通信、同步
BSP
方便差错和性能分析
计算和通信不能重叠

优点：

结构清晰
容易调试
容易分析性能

缺点：

每个相后要同步
计算和通信难以重叠
慢进程会拖住快进程

24. 主从并行 Master-Slave

24.1 基本思想

课件第 25 页讲主从并行。

结构是：

```
Master
├── slave
├── slave
└── slave
```

主进程负责：

串行控制
协调任务
分配工作
收集结果

子进程负责：

计算子任务

24.2 特点

课件列出：

主进程：串行、协调任务
子进程：计算子任务
划分设计技术
与相并行结合
主进程易成为瓶颈

适合任务规模不均、需要动态调度的问题。

缺点是 master 通信量太大时会成为瓶颈。

25. 分治并行 Divide and Conquer

25.1 基本思想

课件第 26 页讲分治并行。

基本结构是：

父进程把负载分割并指派给子进程
子进程继续递归分割
最后归并结果

图中画的是树状分解和向下合并。

25.2 特点

课件列出：

父进程把负载分割并指派给子进程
递归
重点在于归并
分治设计技术
难以负载平衡

分治并行适合：

排序
搜索
快速傅里叶变换
树形计算
递归算法

主要难点是：

子问题规模可能不均
归并阶段可能成为瓶颈

26. 流水线并行 Pipeline

26.1 基本思想

课件第 27 页讲流水线并行。

它由一组阶段组成：

$P1 \rightarrow P2 \rightarrow P3$

每个阶段处理数据的一部分，然后传给下一个阶段。

26.2 特点

课件列出：

一组进程
流水线作业
流水线设计技术

适合：

信号处理
图像处理
编译流水线
矩阵/向量流式计算
生产者-消费者模式

流水线的关键是阶段负载要均衡。

如果 P2 特别慢，那么整个流水线吞吐率就由 P2 决定。

27. 工作池并行 Work Pool

27.1 基本思想

课件第 28 页讲工作池并行。

结构是：

```
Work Pool
├── P1
├── P2
└── P3
```

进程从工作池中取任务执行。

执行过程中还可以产生新任务，再放回池中。

27.2 特点

课件列出：

初始状态：一件工作
进程从池中取任务执行
可产生新任务放回池中
直至任务池为空
易于负载均衡
临界区问题，尤其消息传递

工作池并行适合：

图搜索
分支限界
动态任务树
不规则计算
自适应网格

优点是动态负载均衡好。

缺点是工作池本身需要同步保护，可能出现临界区瓶颈。

28. 计算圆周率的样本程序

28.1 数学公式

课件第 29 页给出用积分计算 π 的公式：

$$\pi = \int_0^1 \frac{4}{(1+x^2)} dx$$

用矩形法近似：

$$\pi \approx \sum_{0 \leq i < N} \frac{4}{(1 + ((i+0.5)/N)^2)} \cdot 1/N$$

其中：

步长 $w = 1/N$
采样点 $x_i = (i + 0.5) / N$

28.2 顺序 C 代码

课件第 30 页给出 C 语言代码：

```
#define N 1000000

main() {
    double local, pi = 0.0, w;
    long i;

    w = 1.0 / N;

    for (i = 0; i < N; i++) {
        local = (i + 0.5) * w;
        pi = pi + 4.0 / (1.0 + local * local);
    }

    printf("pi is %f\n", pi * w);
}
```

28.3 这个程序为什么适合并行化

循环中每个 `i` 的计算基本独立：

```
local = (i + 0.5) * w
term = 4 / (1 + local2)
```

所以可以让多个处理器分别计算一部分 `i`。

最后只需要把局部和做一次归约：

```
pi = sum(local_pi[0..p-1])
```

因此这个例子经常用于说明：

并行循环
任务划分
局部求和
归约
同步

29. 12.2 进程

课件第 31、32 页进入 **12.2 进程**，但这里只给出目录，没有展开具体内容。

29.1 本节目录

课件列出：

- 1 进程的基本概念
- 2 进程的并行执行
- 3 进程的相互作用

可以理解为，这一节准备讨论：

什么是进程
多个进程如何同时执行
进程之间如何通信和同步

29.2 与前文关系

前面讲的 SPMD、MPMD、fork/join、进程编组、交互模式，本质上都以“进程”为基本执行单位。

30. 12.3 线程

课件第 33、34 页进入 **12.3 线程**，也只给出目录。

30.1 本节目录

课件列出：

- 1 线程的基本概念
- 2 线程的管理
- 3 线程的同步

线程比进程更轻量。

一般来说：

进程有独立地址空间
线程共享进程地址空间

所以线程更适合共享内存程序设计，例如 Pthreads 和 OpenMP。

31. 12.4 同步

课件第 35、36 页进入 **12.4 同步**，目录为：

- 1 原子和互斥
- 2 高级同步结构
- 3 低级同步原语

这一节和前面第 16 页内容对应。

31.1 原子和互斥

原子操作保证一段操作不可分割。

互斥保证同一时刻只有一个进程或线程进入临界区。

31.2 高级同步结构

可能包括：

barrier
critical section
monitor
condition variable
event

31.3 低级同步原语

可能包括：

test-and-set
compare-and-swap
fetch-and-add
semaphore
lock/unlock

课件本文件只列出这些小节标题，没有展开具体算法。

32. 12.5 通信

课件第 37、38 页进入 **12.5 通信**，目录为：

- 1 影响通信系统性能的因素
- 2 低级通信支持
- 3 TCP/IP 通信协议组简介

32.1 影响通信系统性能的因素

通常包括：

启动时间
带宽
延迟
协议开销
缓冲区开销
网络拓扑
路由方式
消息大小

这些内容和前面第八章“基本通讯操作”密切相关。

32.2 低级通信支持

低级通信支持通常指系统如何提供 send/receive、DMA、网络接口、缓冲管理等机制。

32.3 TCP/IP 通信协议组

这说明在集群或工作站机群中，通信可能建立在 TCP/IP 协议之上。

但课件本文件没有继续展开细节。

33. 12.6 并行程序设计模型

课件第 39 页开始进入 **12.6 并行程序设计模型**。

第 40、41 页列出四种模型：

隐式并行 Implicit Parallel
数据并行 Data Parallel
共享变量 Shared Variable
消息传递 Message Passing

这四种模型前面在 Part III 中也讲过，这里是从程序设计基础角度再次总结。

34. 隐式并行模型

34.1 基本概况

课件第 42 页讲隐式并行。

基本思想是：

程序员用熟悉的串行语言编程
编译器或运行支持系统自动转化为并行代码

也就是说，程序员不显式写线程、进程、send/receive 或 lock，而是依赖系统自动发现并行性。

34.2 特点

课件列出：

语义简单
可移植性好
单线程，易于调试和验证正确性
效率很低

它的优势是程序员负担小。

缺点是编译器很难自动判断复杂程序中的数据依赖，所以并行效率通常不高。

可以这样记：

隐式并行：程序员最轻松，编译器最困难，效率通常较低。

35. 数据并行模型

35.1 基本概况

课件第 43 页讲数据并行。

它是：

SIMD 的自然模型

主要包含：

局部计算
数据选路操作

数据并行适合数组、矩阵、网格等规则数据结构。

例如：

$C = A + B$

表示对整个数组执行并行加法。

35.2 特点

课件列出：

单线程
并行操作于聚合数据结构，尤其数组
松散同步
单一地址空间
隐式交互作用
显式数据分布

这里的关键是：

程序看起来像单线程
但数组操作在底层并行执行

用户可以指定数据如何分布，但通信通常由系统或编译器处理。

36. 共享变量模型

36.1 基本概况

课件第 44 页讲共享变量模型。

它是以下系统的自然模型：

PVP
SMP
DSM

也就是向量并行机、共享内存多处理机和分布式共享内存系统。

36.2 特点

课件列出：

```
多线程：SPMD，MPMD  
异步  
单一地址空间  
显式同步  
隐式数据分布  
隐式通信
```

核心是：

```
多个线程通过读写共享变量通信
```

程序员不用显式发送消息，但必须显式处理同步问题。

例如：

```
lock(S);  
sum = sum + local;  
unlock(S);
```

共享变量模型的主要风险是：

```
数据竞争  
死锁  
临界区开销  
false sharing  
同步等待
```

37. 消息传递模型

37.1 基本概况

课件第 45 页讲消息传递模型。

它是以下系统的自然模型：

```
MPP  
COW
```

也就是大规模并行处理机和 workstation 机群。

37.2 特点

课件列出：

```
多线程  
异步  
多地址空间  
显式同步  
显式数据映射和负载分配  
显式通信
```

核心是：

```
每个进程有自己的地址空间  
进程之间通过消息传递交换数据
```

典型形式是：

```
send(data, dest);  
recv(data, source);
```

MPI 就是典型消息传递模型。

37.3 与共享变量模型对比

项目	共享变量模型	消息传递模型
地址空间	单一地址空间	多地址空间
通信方式	读写共享变量	send/receive
数据分布	隐式	显式
同步	显式	显式
编程难点	数据竞争、锁	通信匹配、死锁、数据划分
适合机器	SMP, DSM	MPP, Cluster

可以简单记：

共享变量：通信隐式，同步显式
消息传递：通信显式，同步显式

38. 本章核心总结

38.1 这一章讲什么

PC12.pdf 的核心可以概括为：

并行程序设计基础

- └— 为什么难
- └— 并行语言怎么构造
- └— 并行性如何表达
- └— 进程如何交互
- └— 常见编程风范
- └— 四种编程模型

38.2 最重要的知识线索

这一章的主线是：

并行程序 = 多个执行单元 + 数据划分 + 通信 + 同步 + 映射

也就是说，并行程序不仅要“同时执行”，还要解决：

谁执行？
执行什么？
数据在哪里？
什么时候通信？
什么时候同步？
如何避免冲突？
如何保持负载均衡？

38.3 五种并行编程风范

相并行：超级步，计算与同步交替
主从并行：**master** 分配任务，**slave** 计算
分治并行：递归分解，重点在归并
流水线并行：阶段串接，数据流动
工作池并行：动态取任务，适合负载均衡

38.4 四种并行程序设计模型

隐式并行：自动并行化，简单但效率低
数据并行：数组整体操作，适合 SIMD/科学计算
共享变量：单地址空间，隐式通信，显式同步
消息传递：多地址空间，显式通信，适合 MPP/Cluster

39. 复习重点

这一章最应该掌握：

1. 并行程序设计难在哪里
2. 三种并行语言构造方法：库例程、扩展语言、编译注释
3. SIMD、MIMD、SPMD、MPMD 的区别
4. 为什么大规模并行程序常用 SPMD
5. 静态并行性和动态并行性的区别
6. Fork/Join 的作用
7. 划分与分配的区别
8. 并行度和并行粒度的关系
9. 通信、同步、聚集的区别
10. 点到点、广播、播撒、收集、归约、扫描、全交换、移位
11. 五种并行编程风范
12. 四种并行程序设计模型

一句话概括：

第十二章是在讲并行程序设计的“基本语言和组织方式”：并行程序要处理进程/线程、划分/分配、通信/同步和编程模型；真正写好并行程序，关键不是只会启动多个进程，而是要让计算多、通信少、同步少、负载均衡、结构清晰。